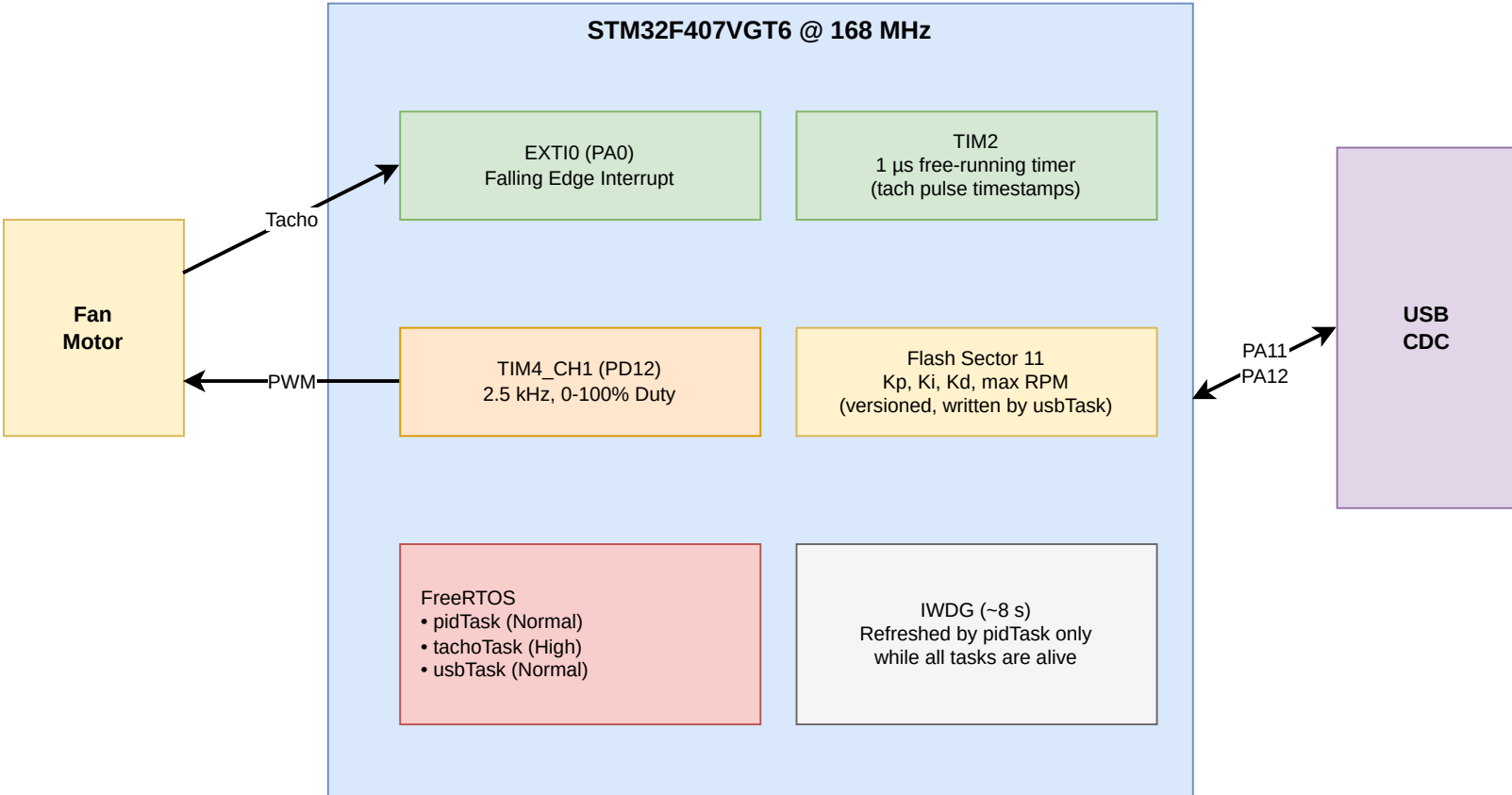
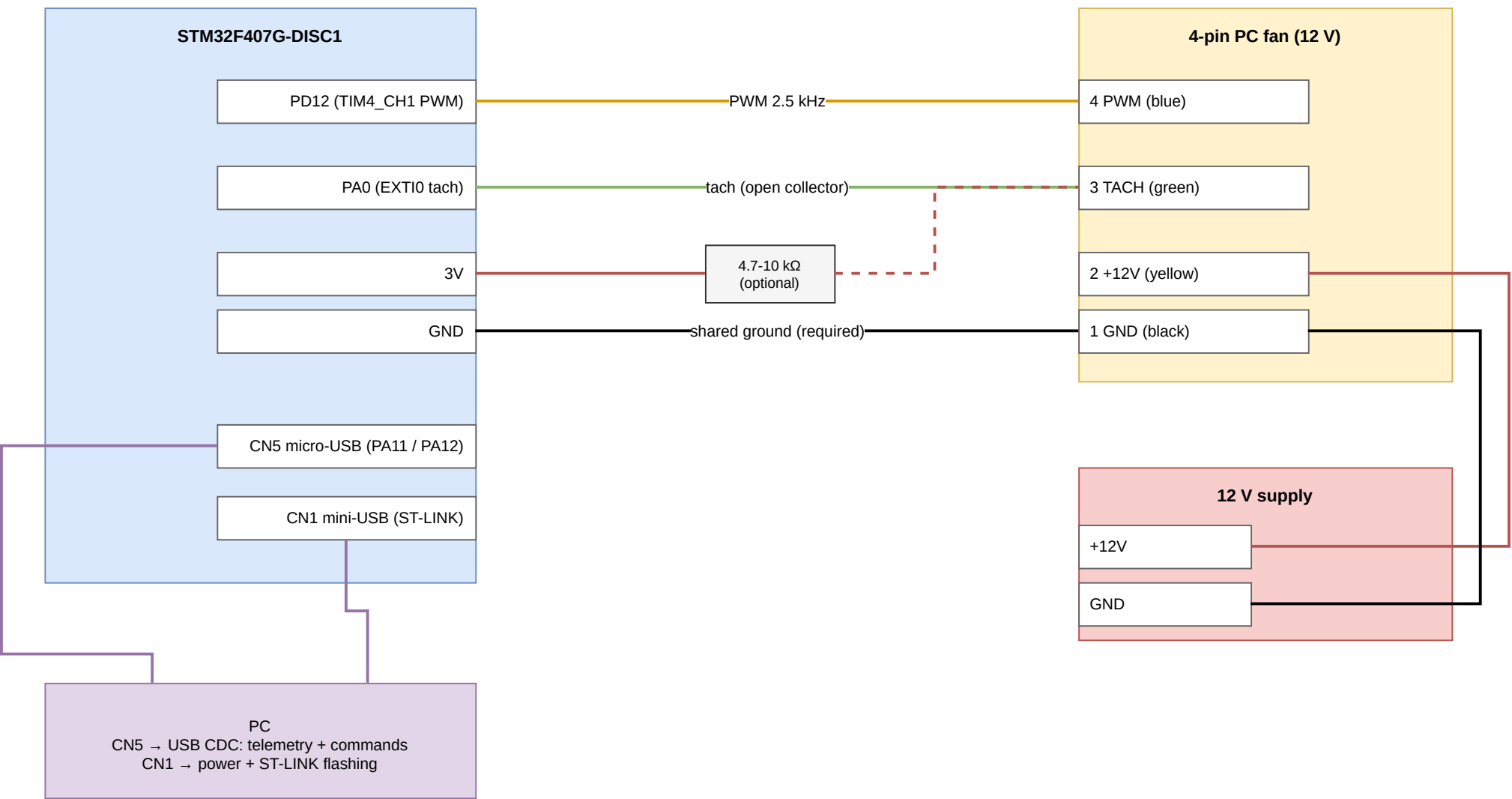


STM32 Fan Control System Architecture



Fan Controller Pinout & Wiring

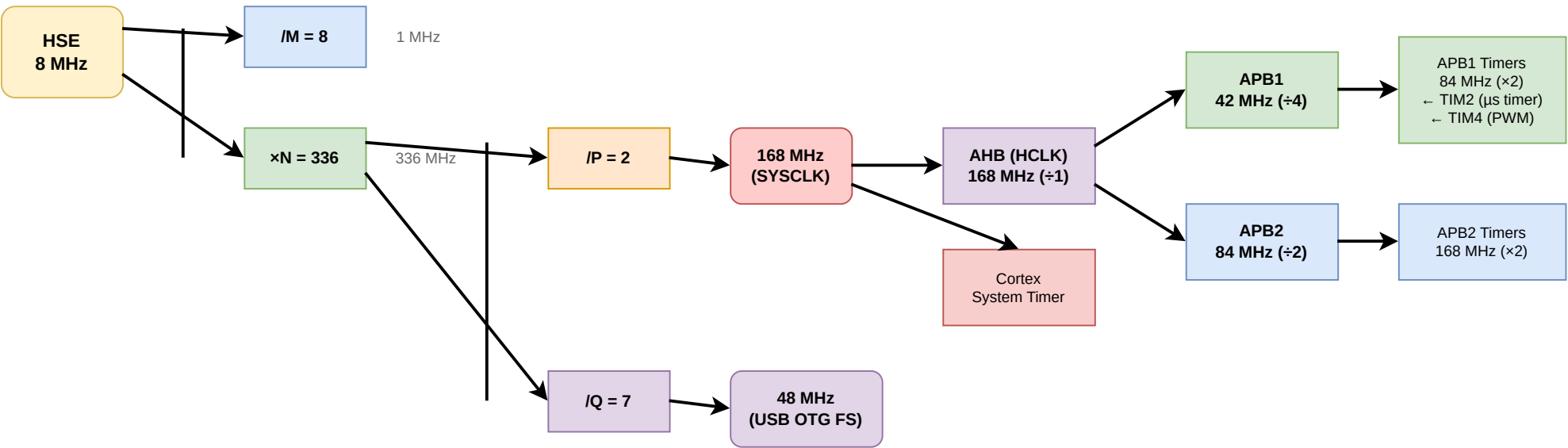


PIN ASSIGNMENT

Pin	Function	Peripheral / setting
PD12	Fan PWM output (also green LED LD4)	TIM4_CH1, PWM mode 1, 2.5 kHz, ARR 399
PA0	Fan tach input (also blue USER button)	EXTI0 falling edge, internal pull-up, NVIC 5
PA11 / PA12	USB OTG FS DM / DP	CDC virtual COM port, 48 MHz from PLL Q
PD13	Orange LED	on after a watchdog reset
PD14	Red LED	on in Error_Handler
—	TIM2 (internal)	1 μs free-running counter for tach timestamps

Never connect 12 V to the board. The tach pull-up goes to 3V, not 5V.

System Clock Configuration



PLL Configuration:
HSE = 8 MHz
M = 8, N = 336, P = 2, Q = 7
VCO = 336 MHz
SYSCLK = 168 MHz
USB = 48 MHz

Peripheral Configuration

TIM2 - Microsecond Timer (Tachometer Timing)

Clock Source: APB1 Timer (84 MHz)
Prescaler: 83
Counter Freq: $84\text{MHz} / (83+1) = 1\text{ MHz} \rightarrow 1\mu\text{s}$ resolution
Mode: 32-bit free-running
ARR: 0xFFFFFFFF (max)

TIM4 - PWM Generation (Fan Control)

Clock Source: APB1 Timer (84 MHz)
Prescaler: 83
Timer Freq: $84\text{MHz} / (83+1) = 1\text{ MHz}$
ARR (Period): 399
PWM Freq: $1\text{MHz} / (399+1) = 2.5\text{ kHz}$
Channel: CH1 (PD12)
Mode: PWM Mode 1
Duty Cycle: CCR1 / 400 × 100%

USB OTG FS - CDC Virtual COM Port

Pins: PA11 (DM), PA12 (DP)
Clock: 48 MHz (PLL /Q=7)
Class: CDC (Virtual COM Port)
Buffer: TX: 2048 bytes, RX: 2048 bytes
Commands: s<rpm>, m<pwm>, p/i/d<val>, r<rpm>, save

GPIO - Tachometer Input

Pin: PA0
Mode: External Interrupt (EXTI0)
Trigger: Falling Edge
Pull: Pull-up
Priority: 5

Configuration Summary:

Timers:

- TIM2: Microsecond timestamp counter for tachometer pulse timing
- 32-bit counter prevents overflow for ~71 minutes
- 1 μs resolution sufficient for RPM measurement (740-2830 RPM)

- TIM4: PWM generation for fan speed control
- 2.5 kHz, ARR 399 $\rightarrow$ 100% duty when CCR > ARR
- 400 steps (0-399) provides 0.25% duty cycle resolution

Communication:

- USB CDC: Virtual COM port for real-time telemetry and control
- No UART-to-USB adapter required
- 2KB buffers handle burst telemetry at 4 Hz

Interrupt:

- EXTI0: Falling edge detection on tachometer signal
- NVIC priority 5 (highest allowed for FreeRTOS API calls)
- Internal pull-up (~40 k Ω) on the open-collector tach; optional 4.7-10 k Ω to 3 V

Reliability:

- IWDG: ~8 s timeout, refreshed only while all three tasks check in
- Flash sector 11 (0x080E0000): settings record, excluded from the linker FLASH region

Task Priorities & Synchronization

EXTI0_IRQHandler (tach falling edge, NVIC 5)

1. tach_capture_us = TIM2 counter
2. osSemaphoreRelease(tachoSem)

OTG_FS ISR → process_usb_command (NVIC 5)

1. command_line_feed() + command_parse()
2. write setpoint / manual / gains / max_rpm
3. save or invalid → osThreadFlagsSet(usbTask)

tachoTask (osPriorityHigh, event-driven)

```
for (;;) {  
    heartbeat_tacho_ms = now;  
    if (osSemaphoreAcquire(tachoSem, 1000 ms) == osOK) {  
        flash_busy? → discard pulse, next one becomes the reference  
        first pulse? → reference only  
        else → measured_rpm = tach_rpm_from_period(Δt) // rejects > 10,000 RPM  
    } else → measured_rpm = 0, tach_timeout = true  
}
```

pidTask (osPriorityNormal, 40 Hz)

```
for (;;) {  
    osDelayUntil(tick += 25 ms); // resync if a deadline was missed  
    watchdog_service(); // refresh IWDG only if all heartbeats < 3 s old  
    in = read_control_inputs(); // interrupts off: consistent snapshot  
    tach_timeout? → pid_reset()  
    in.bumpless && !in.manual → pid_bumpless_transfer()  
    output = in.manual ? in.manual_pwm : pid_update(FF + P + I + D, clamp 10-95 %);  
    pid_output = output; set_fan_speed(output);  
}
```

usbTask (osPriorityNormal, 4 Hz telemetry)

```
for (;;) {  
    heartbeat_usb_ms = now;  
    flags = osThreadFlagsWait(SAVE | ERROR, timeout = time to next telemetry);  
    ERROR → "ERR: invalid command"  
    SAVE → flash_busy = true; settings_save(); flash_busy = false; → "OK: saved"  
    timeout → telemetry line every 250 ms  
} // only caller of CDC_Transmit_FS
```

Priorities

- ISRs (NVIC 5): preempt every task
- tachTask: High, so pulses are processed before pid/usb run
- pidTask, usbTask: Normal

Synchronization

- Semaphore: EXTI0 ISR → tachTask
- Thread flags: USB ISR → usbTask (save, errors)
- PRIMASK snapshot: USB ISR → pidTask inputs
- Single 32-bit reads/writes: task → task

Timing

- tachTask: per pulse (~20 ms at 1500 RPM), 1000 ms timeout = fan stopped
- pidTask: 25 ms fixed rate (osDelayUntil)
- usbTask: 250 ms fixed rate
- Flash save: CPU stalls ~1-2 s; pidTask resyncs, tachTask discards the delayed pulse

Watchdog

- IWDG ~8 s, started just before the scheduler
- pidTask refreshes it only while tachTask and usbTask heartbeats are < 3 s old
- A hang in any task or the scheduler → reset; next boot prints a WARN and lights the orange LED

semaphore

measured_rpm, tach_timeout

pid_output

PRIMASK
snapshot

thread
flags

Race Condition & Thread Safety

THE PROBLEM: the USB ISR changes several inputs while pidTask is reading them

Example: the fan is in manual mode and the user types "s1600"

t	pidTask	USB ISR ("s1600")
t1	in.manual = manual_mode; → true	
t2		manual_mode = false; pending_bumpless_transfer = true;
t3	in.bumpless = pending_bumpless_transfer; → true pending_bumpless_transfer = false;	
t4	in.manual is true → bumpless transfer skipped, flag already cleared → lost	

Result: next cycle runs in auto mode with the old integral → PWM jumps (a "bump")

Why osKernelLock() does not fix it: it only stops task switches. Interrupts still run, and the writer here is an ISR.

THE SOLUTION: read every ISR-written input in one interrupts-off snapshot

```
static control_inputs_t read_control_inputs(void)
{
    control_inputs_t in;
    uint32_t primask = __get_PRIMASK();
    __disable_irq();           // ISRs cannot run in between
    in.manual      = manual_mode;
    in.bumpless    = pending_bumpless_transfer;
    pending_bumpless_transfer = false;    // read-and-clear is now atomic
    in.setpoint    = setpoint_rpm;
    in.manual_pwm  = manual_pwm;
    in.gains       = { Kp, Ki, Kd };
    __set_PRIMASK(primask);    // restore previous state
    return in;
}
```

Why this works

- Interrupts are off for ~10 load/store instructions (well under 1 μs at 168 MHz)
- pidTask then works only on its local copy, so the ISR can change the globals at any time

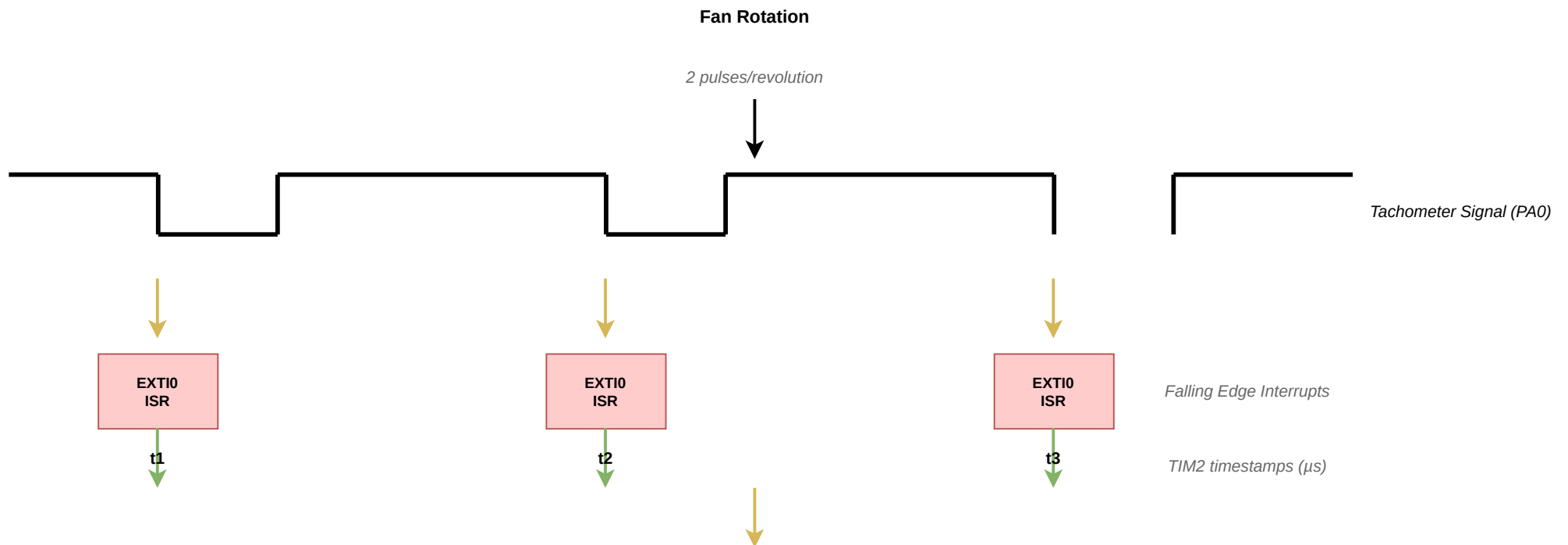
Why the other shared variables need no lock

- Cortex-M4 reads and writes an aligned 32-bit word (float, uint32_t) or a byte in one instruction
- Each is written from a single task or ISR
- The PID state (integral, last_measured) is a local in pidTask, not shared at all

USAGE MAP

Variable	Writer	Readers	Protection
setpoint_rpm, manual_mode, manual_pwm, Kp/Ki/Kd, pending_bumpless	USB ISR	pidTask (usbTask: setpoint)	PRIMASK snapshot
max_rpm	USB ISR	usbTask (save)	single 32-bit word
save / invalid-command request	USB ISR	usbTask	thread flags
tacho_capture_us	EXTI0 ISR	tachoTask	single word + semaphore
measured_rpm	tachoTask	pidTask, usbTask	single 32-bit word
tacho_timeout	tachoTask (set)	pidTask (clear)	single byte
pid_output	pidTask	usbTask	single 32-bit word
flash_busy	usbTask	tachoTask	single byte
heartbeat_tacho_ms / heartbeat_usb_ms	tachoTask / usbTask	pidTask (watchdog)	single 32-bit word
integral, last_measured	pidTask	—	not shared (local)

Tachometer to RPM Conversion



Example Calculation:

For 1500 RPM fan:

- Δt between pulses = 20,000 μs
- $RPM = 60,000,000 / (20,000 \times 2)$
- $RPM = 60,000,000 / 40,000$
- $RPM = 1500$

Notes:

- TIM2 clock 84 MHz (APB1 timer)
- Prescaler 83 ($\div 84$) $\rightarrow$ 1 μs
- 2 pulses per revolution
- First pulse only sets the reference
- 1000 ms timeout $\rightarrow$ fan stopped
- Δt is wrap-safe (uint32)

tachoTask Processing

$$\Delta t = t2 - t1$$

$$RPM = \frac{60,000,000}{\Delta t \times 2}$$

Timeout (1000ms)?

- └─ Yes: $RPM = 0$
- └─ No: Update measured_rpm

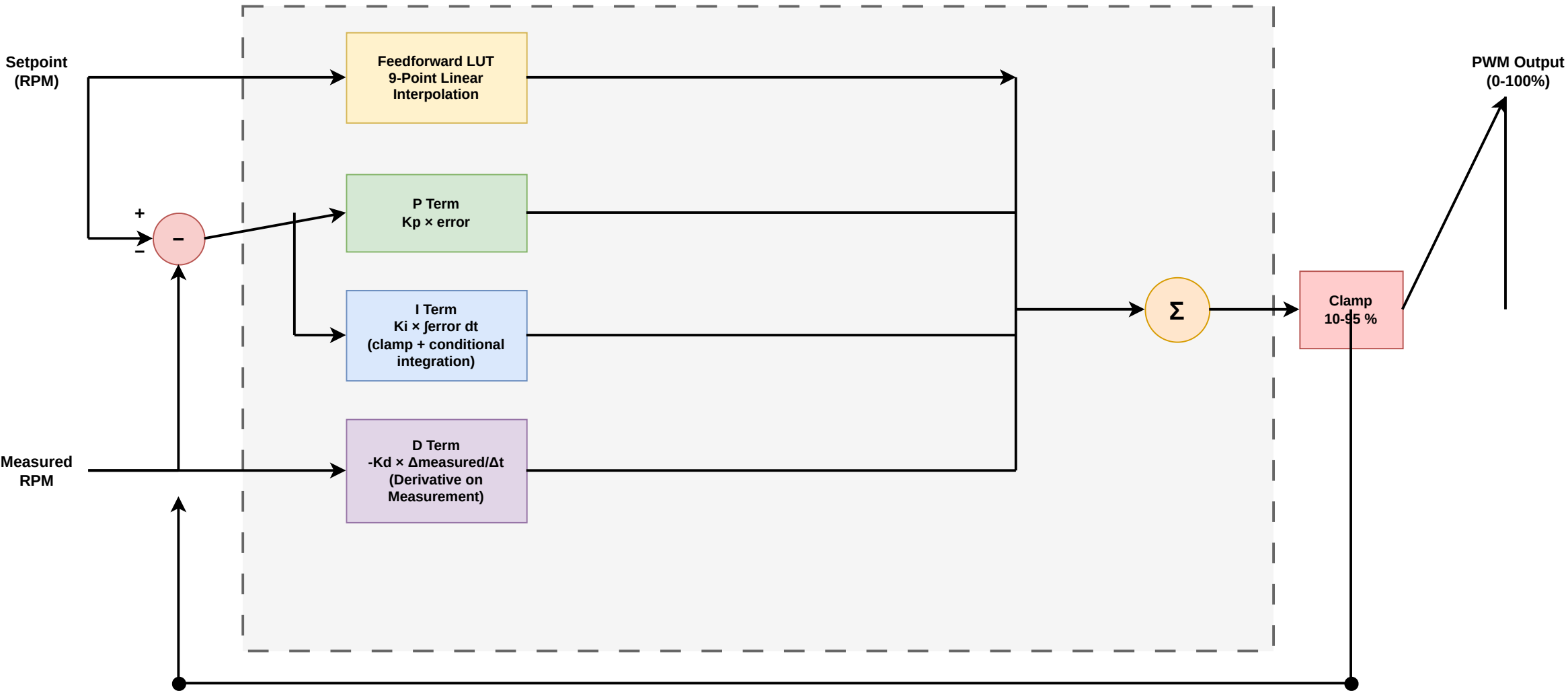
First pulse after startup?

- └─ Ignore (rejected)

RPM > 10,000? └─ Reject as noise

Pulse during flash save? └─ Discard, next pulse becomes the reference

PID Control Architecture (40 Hz Loop)



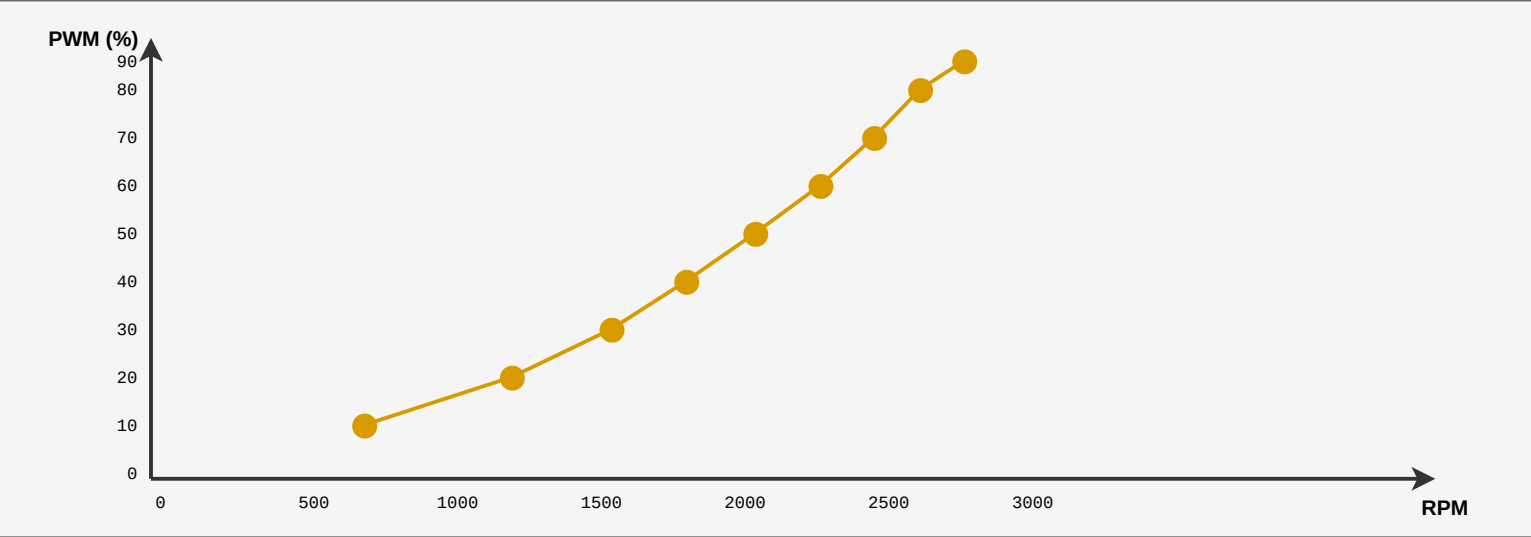
- Notes:
- Sample Time: 25 ms (40 Hz); output is 0 when the setpoint is 0
 - Anti-windup 1: integral clamped to $[(\text{MIN} - \text{FF}) / K_i, (\text{MAX} - \text{FF}) / K_i]$
 - Anti-windup 2: integral frozen while the output is saturated in the direction of the error
 - Derivative on measurement prevents setpoint kick; skipped across 0 RPM
 - Manual → auto: integral back-calculated for bumpless transfer
 - FF LUT: 740-2830 RPM → 10-90% PWM

Feedforward Lookup Table

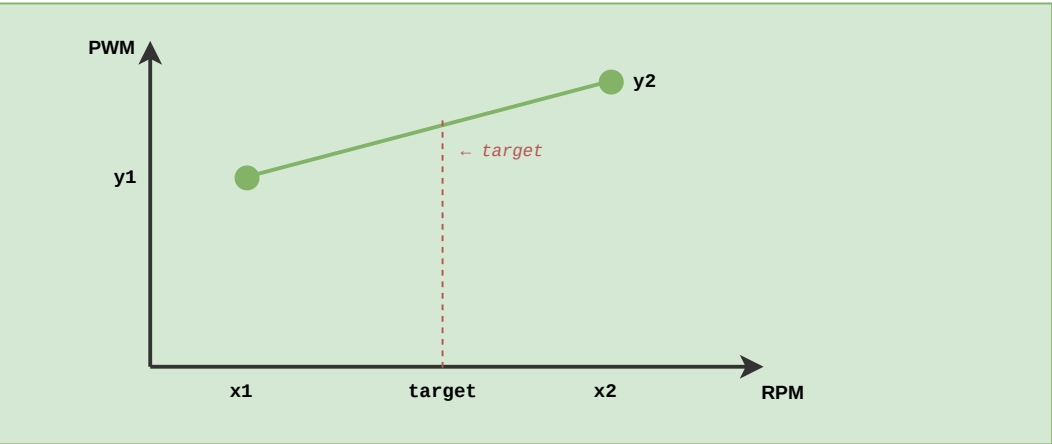
CALIBRATION DATA (Open-Loop Measurement)

Pt	RPM	PWM (%)
1	740	10
2	1250	20
3	1600	30
4	1860	40
5	2100	50
6	2325	60
7	2515	70
8	2675	80
9	2830	90

GRAPHICAL REPRESENTATION



LINEAR INTERPOLATION



Formula:

$$y = y1 + \frac{y2 - y1}{x2 - x1} \times (target - x1)$$

IMPLEMENTATION

```
const float ff_rpm_points[] = {740, 1250, 1600, 1860, 2100, 2325, 2515, 2675, 2830};
const float ff_pwm_points[] = {10.0, 20.0, 30.0, 40.0, 50.0, 60.0, 70.0, 80.0, 90.0};

float pid_feedforward(float rpm) {
    if (rpm <= ff_rpm_points[0]) return ff_pwm_points[0];
    if (rpm >= ff_rpm_points[8]) return ff_pwm_points[8];

    for (int i = 0; i < 8; i++) {
        if (rpm >= ff_rpm_points[i] && rpm <= ff_rpm_points[i+1]) {
            float x0 = ff_rpm_points[i];
            float y0 = ff_pwm_points[i];
            float x1 = ff_rpm_points[i+1];
            float y1 = ff_pwm_points[i+1];
            return y0 + (rpm - x0) * (y1 - y0) / (x1 - x0);
        }
    }
    return 0.0f;
}
```

INTEGRATION WITH PID

```
float ff_term = pid_feedforward(setpoint_rpm); // most of the output at steady state
float p_term = Kp * error;
float i_term = Ki * integral;
float d_term = Kd * (-delta_measured / dt);

float output = ff_term + p_term + i_term + d_term; // Combine
```

ANTI-WINDUP WITH FEEDFORWARD

```
// 1. Conditional integration: freeze the integral while the output is
//    already saturated in the direction of the error
float candidate = integral + error * dt;
float unsaturated = ff_term + p_term + Ki * candidate + d_term;
bool winding_up = (unsaturated > PID_OUTPUT_MAX && error > 0) ||
                  (unsaturated < PID_OUTPUT_MIN && error < 0);
if (!winding_up) integral = candidate;

// 2. Clamp: FF + Ki*integral must stay inside the output range
//    e.g. FF = 57% → Ki*integral ∈ [10 - 57, 95 - 57] = [-47%, +38%]
integral = clampf(integral, (PID_OUTPUT_MIN - ff_term) / Ki,
                  (PID_OUTPUT_MAX - ff_term) / Ki);
```

EXAMPLE: RPM = 1800

Between Point 3 (1600, 30%) and Point 4 (1860, 40%)

PWM = 30.0 + (1800 - 1600) × (40.0 - 30.0) / (1860 - 1600)
= 30.0 + 200 × 10.0 / 260
= 37.69%

Result: 37.69% PWM